PRODUCT REQUIREMENTS DOCUMENT

House Price Prediction

Machine Learning — Regression Project

Dataset: Ames Housing (Dean De Cock, 2011)

Document Title	House Price Prediction — Product Requirements Document
Version	1.0
Date	May 7, 2026
Author	AI Engineering Student
Status	Draft
Project Type	Machine Learning — Regression / Tabular Data
Dataset	Ames Housing Dataset (Dean De Cock, 2011)

1. Project Overview

This project builds a machine learning system that predicts residential house sale prices from 79 property features. It is a regression problem — predicting a continuous dollar value — and demonstrates core skills in data cleaning, feature engineering, model selection, hyperparameter tuning, model interpretability, and web deployment. These are the skills most commonly tested in AI engineering interviews and Kaggle competitions.

1.1 Objectives

- Accurately predict house sale prices using the Ames Housing dataset (target: RMSE < $25,000 on test set).
- Implement and compare four ML models: Linear Regression, Ridge/Lasso, Random Forest, and XGBoost.
- Engineer at least 6 derived features from raw columns to improve model performance.
- Correctly handle the 3 distinct types of missing values as defined in the dataset's data dictionary.
- Apply SHAP values to generate per-prediction explanations of what drove each price estimate.
- Deploy the model as an interactive Streamlit web app where a user can input property details and receive an instant price prediction with a confidence range.

1.2 Scope

In scope: full data pipeline from raw CSV to deployed Streamlit app, covering EDA, feature engineering, model training and comparison, hyperparameter tuning, SHAP interpretability, and model serialization. Out of scope: scraping live real estate listings, multi-city generalization, automated retraining, and deep learning approaches (stretch goals for v2).

2. Dataset — Ames Housing

2.1 Dataset Overview

The Ames Housing dataset was compiled by Dean De Cock (2011) as a modern, richer alternative to the classic Boston Housing dataset. It documents residential property sales in Ames, Iowa, USA, between 2006 and 2010. It is the standard benchmark dataset for regression projects and is hosted on Kaggle as the 'House Prices: Advanced Regression Techniques' competition.

- 2,930 residential house sale records
- 79 explanatory features + 1 target variable (SalePrice)
- Sales price range: $12,789 to $755,000 (right-skewed distribution)
- Time period: January 2006 – July 2010
- Source: Kaggle — kaggle.com/c/house-prices-advanced-regression-techniques

2.2 Feature Breakdown

Feature Group	Count	Key Examples
Numerical	36	LotArea, GrLivArea, TotalBsmtSF, GarageArea, YearBuilt
Categorical (nominal)	23	Neighborhood, HouseStyle, BldgType, Foundation, SaleType
Categorical (ordinal)	21	OverallQual, ExterQual, KitchenQual, BsmtCond (Ex/Gd/TA/Fa/Po)
Target variable	1	SalePrice — continuous $USD, range $12,789–$755,000

2.3 Top Features by Correlation to SalePrice

Feature	Correlation (r)	Notes
OverallQual — overall material & finish quality	0.79	Strongest predictor; rated 1–10 scale
GrLivArea — above-ground living area (sq ft)	0.71	Key size metric; engineer price/sqft from this

GarageCars — garage capacity in car count	0.64	Proxy for total garage quality and size
TotalBsmtSF — total basement area (sq ft)	0.61	Combine BsmtFinSF1, BsmtFinSF2, BsmtUnfSF
1stFlrSF — first floor area (sq ft)	0.61	Highly correlated with TotalBsmtSF
YearBuilt — original construction year	0.52	Engineer HouseAge = 2010 - YearBuilt
FullBath — full bathrooms above grade	0.56	Combine with HalfBath for TotalBaths feature

2.4 Target Variable — SalePrice

SalePrice is a right-skewed continuous variable. The majority of houses fall between $100,000 and $300,000, but a long tail of high-value properties skews the distribution. This has two important implications:

- Apply log1p(SalePrice) before training and expm1() on predictions. This normalizes the distribution, reduces the influence of extreme values, and improves model performance across all algorithms.
- RMSE computed on log-transformed predictions (log-RMSE) is the standard metric for this dataset and matches the Kaggle leaderboard scoring.

2.5 Missing Value Strategy

The Ames dataset contains 19 columns with missing values. These are NOT all the same — they fall into three distinct categories that require different imputation strategies:

Type	Examples	Strategy
Meaningful NaN — feature doesn't apply	PoolQC, GarageType, BsmtCond	Fill with string 'None' — absence is informative
True missing — data not collected	LotFrontage (259 NaN)	Impute with median of neighbourhood group
Linked numeric — tied to absent feature	GarageArea, BsmtFinSF1	Fill with 0 — no garage means 0 sq ft

Additionally, 2 extreme outlier houses (GrLivArea > 4,000 sq ft with very low sale prices) are documented in De Cock's paper and should be removed before training to prevent distortion of the regression line.

3. Feature Engineering Requirements

Raw features alone are insufficient for a high-performing model. The following engineered features must be created as part of the preprocessing pipeline:

1. TotalSF — total_bsmt_sf + 1st_flr_sf + 2nd_flr_sf. Combines all interior floor area into a single total square footage metric, which is the most commonly used metric in real estate.
2. HouseAge — YrSold - YearBuilt. Converts an absolute year into a relative age at time of sale, which has a stronger linear relationship with price than the raw year.
3. RemodAge — YrSold - YearRemodAdd. Age since last remodel; captures renovation recency which is a key price driver.
4. TotalBaths — FullBath + 0.5 * HalfBath + BsmtFullBath + 0.5 * BsmtHalfBath. Aggregates all bathroom counts into a single weighted score.
5. HasPool — binary flag (PoolArea > 0). Presence of a pool is more predictive than pool area for most price ranges.
6. TotalPorchSF — sum of OpenPorchSF, EnclosedPorch, 3SsnPorch, ScreenPorch. Aggregates all porch types into a single outdoor living area metric.
7. PricePerSqFt — (not used as a feature; used for EDA visualization and outlier detection).

4. Functional Requirements

ID	Requirement	Description	Priority
FR-01	Data ingestion	Load train.csv and test.csv from the data/ directory into pandas DataFrames.	Must have
FR-02	Outlier removal	Remove 2 documented outlier records (GrLivArea > 4000 sq ft, SaleCondition not Normal).	Must have
FR-03	Missing value imputation	Apply 3-strategy imputation: 'None' for meaningful NaN, median for true missing, 0 for linked numeric.	Must have
FR-04	Target transformation	Apply log1p() to SalePrice at train time; apply expm1() to predictions before evaluation.	Must have
FR-05	Feature engineering	Create 6 derived features as specified in Section 3.	Must have
FR-06	Encoding	One-hot encode nominal categoricals; ordinal-encode quality/condition scales (Po=1 to Ex=5).	Must have
FR-07	Feature scaling	Apply StandardScaler to numerical features for linear models; tree models do not require scaling.	Must have
FR-08	Pipeline wrapping	Wrap all preprocessing and model	Must have

		steps in a single sklearn Pipeline or ColumnTransformer to prevent leakage.	
FR-09	Model training	Train Linear Regression, Ridge, Random Forest, and XGBoost with 5-fold cross-validation.	Must have
FR-10	Evaluation report	Report RMSE, MAE, and R^2 for each model on the held-out test set. Include log-RMSE.	Must have
FR-11	Model serialization	Save best model to models/house_price_model.joblib.	Must have
FR-12	SHAP analysis	Compute SHAP values for best model; generate summary plot and per-prediction waterfall chart.	Should have
FR-13	Streamlit app	Build web app where user inputs 10 key property features and receives price estimate + confidence range.	Should have
FR-14	Kaggle submission	Generate submission.csv in Kaggle format from test.csv predictions.	Should have
FR-15	Model stacking	Implement a stacked ensemble (Ridge meta-learner over RF + XGBoost base) for improved accuracy.	Nice to have

5. Non-Functional Requirements

5.1 Performance Targets

- Log-RMSE on test set: target < 0.12 (top-15% on Kaggle leaderboard for this competition).
- MAE on original price scale: target < $18,000.
- R^2 on test set: target > 0.90.
- Training time: full pipeline including cross-validation < 5 minutes on a standard laptop CPU.
- Streamlit app response: prediction returned in < 2 seconds.

5.2 Code Quality

- All source files organized into modules (data.py, features.py, train.py, evaluate.py, app.py).
- Each function includes a Google-style docstring with Args and Returns sections.
- No magic numbers in code — all thresholds and parameters defined as named constants.

- Unit tests cover at minimum: imputation logic, feature engineering outputs, and pipeline fit/predict cycle.

6. Technical Stack

6.1 Core Libraries

- Python 3.10+
- pandas — data loading, EDA, feature engineering
- numpy — numerical operations
- scikit-learn — Pipeline, ColumnTransformer, LinearRegression, Ridge, Lasso, RandomForestRegressor, cross_val_score, metrics
- xgboost — XGBRegressor (primary candidate for best model)
- shap — SHAP value computation and visualizations
- joblib — model serialization
- streamlit — interactive web application
- matplotlib / seaborn — EDA visualizations

6.2 Project Structure

```
house-price-predictor/ ├── data/ │   ├── train.csv │   ├── test.csv │   └──
data_description.txt ├── notebooks/ │   ├── 01_eda.ipynb │   └──
02_feature_analysis.ipynb ├── src/ │   ├── data.py          # loading & outlier
removal │   ├── features.py      # imputation & feature engineering │   ├── train.py
# model training & serialization │   ├── evaluate.py      # metrics & SHAP analysis |
└── app.py           # Streamlit web app ├── models/ │   └── house_price_model.joblib
├── tests/ │   └── test_pipeline.py ├── requirements.txt └── README.md
```

7. Project Milestones

Estimated duration: 7 weeks at a part-time student pace (10–15 hours per week).

Phase	Milestone	Duration	Deliverable
1	Data acquisition & EDA	1 week	Cleaned dataset + EDA notebook with correlation heatmap
2	Feature engineering pipeline	1 week	sklearn ColumnTransformer with 6+ engineered features
3	Baseline & model training	1 week	4 trained models with cross-val RMSE scores
4	Hyperparameter tuning	1 week	Best model with GridSearch/Optuna results

5	Interpretability (SHAP)	1 week	SHAP summary plot + per-prediction explanations
6	Streamlit app deployment	1 week	Live web app with form input and price estimate
7	Documentation & portfolio	1 week	README + demo video + Kaggle submission

8. Evaluation Criteria & Acceptance Tests

8.1 Model Acceptance Criteria

- Best model achieves log-RMSE < 0.12 on held-out test set.
- All 4 models trained, evaluated, and compared in a side-by-side table.
- Cross-validation score reported (5-fold mean ± std) for each model.
- SHAP summary plot identifies top 10 most important features.

8.2 Application Acceptance Criteria

- Streamlit app accepts input for: GrLivArea, OverallQual, TotalBsmtSF, GarageCars, YearBuilt, Neighborhood, BldgType, FullBath, BedroomAbvGr, KitchenQual.
- App displays predicted price, log-scale confidence interval, and a mini SHAP waterfall chart for the prediction.
- App handles invalid inputs gracefully with clear error messages.

9. Risks & Mitigations

Risk	Impact	Mitigation
Incorrect handling of meaningful NaN values	High	Follow De Cock's data dictionary; fill 'None' vs 0 vs median by type
Data leakage from fitting scaler/encoder on full data	High	Wrap everything in sklearn Pipeline; fit only on X_train
Outlier houses distort regression line	Medium	Remove 2 known large area outliers (GrLivArea > 4000, SaleCondition = Partial)
Target skewness inflates RMSE on raw scale	Medium	Apply log1p() to SalePrice; reverse with expm1() on predictions
Multicollinearity between size features	Medium	Use Ridge/Lasso regularization; drop redundant features after VIF analysis

10. Stretch Goals

The following enhancements will significantly elevate the portfolio project for senior roles or Kaggle competition placement:

8. Stacked ensemble — train Ridge as a meta-learner on out-of-fold predictions from Random Forest + XGBoost. Typically reduces log-RMSE by 0.005–0.015.
9. Optuna hyperparameter optimization — replace GridSearchCV with Optuna for more efficient Bayesian search over XGBoost hyperparameters.
10. Neighbourhood-level median imputation — instead of global median for LotFrontage, use the neighbourhood group median for more accurate imputation.
11. Docker deployment — containerize the Streamlit app and deploy to Railway or Render for a permanent public URL.
12. Kaggle submission — submit predictions and document leaderboard position in the README.

11. Portfolio Presentation Notes

When presenting this project in interviews or on GitHub, emphasize these points:

- The 3-strategy missing value approach — demonstrates that you read the data dictionary, not just ran df.fillna(0).
- log1p target transformation — shows understanding of regression assumptions and skewness.
- sklearn Pipeline — signals production-grade thinking; prevents leakage, enables clean deployment.
- SHAP explanations — demonstrates awareness of responsible/explainable AI, not just accuracy chasing.
- Streamlit app — converts a notebook into usable software; interviewers can actually try it.

Recommended GitHub README sections: Project Summary, Live Demo link, Model Performance Table (all 4 models), SHAP Plot screenshot, Setup & Usage instructions, Key Engineering Decisions.

This PRD is a living document. Update version and date when requirements change.